PASOS PARA DESARROLLAR TERRAFORM

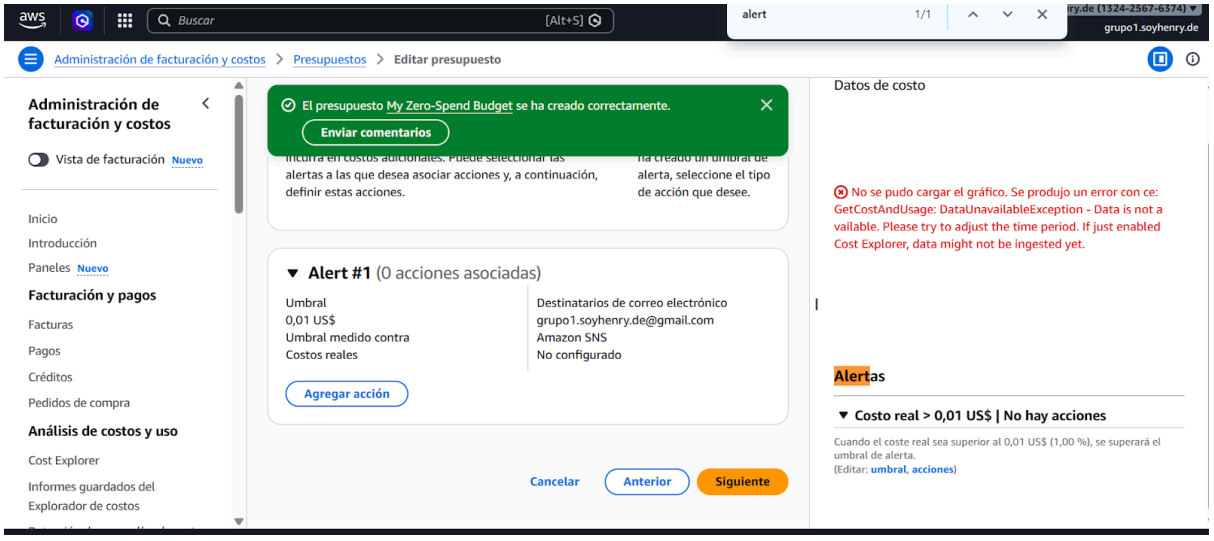
1) Instalar Terraform: En Git-Bash: winget install HashiCorp.Terraform
2) terraform version
3) Ingresar a la carpeta Proyecto-Latam-Turismo-Sustentable\infra
4) Dentro de ella crear el archivo budget.tf
5) Llenar con los procesos a realizar
6) terraform init
7) terraform plan: Muestra la planificación de lo que va a hacer
8) terraform apply: lo ejecuta

Para ver las credenciales
aws sts get-caller-identity

aws configure --profile grupo1
para configurar las credenciales

- **AWS Access Key ID**: la de `adrian.sosa`
- **AWS Secret Access Key**: la de `adrian.sosa`
- **Default region name**: `us-east-1`
- **Default output format**: podés poner `json`

Alerta de Budget creada

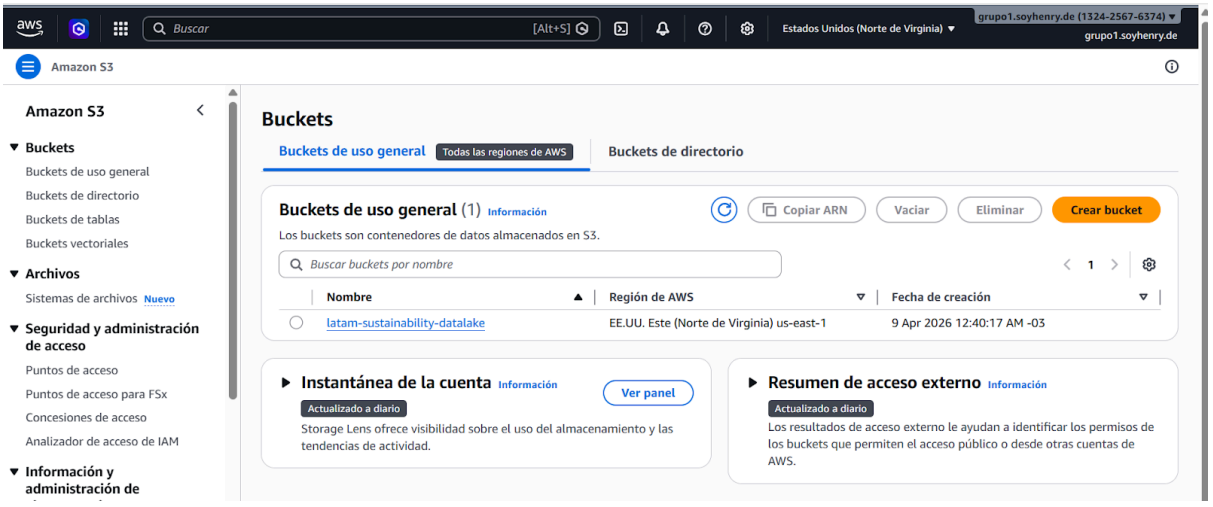


CREACIÓN DEL BUCKET S3

```
$ terraform plan
module.s3.aws_s3_bucket.bronze: Refreshing state...
[id=latam-sustainability-bronze]
aws_budgets_budget.zero_budget: Refreshing state...
[id=132425676374:zero-budget]
```

```
Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.s3.aws_s3_bucket.datalake will be created
  + resource "aws_s3_bucket" "datalake" {
      + acceleration_status         = (known after apply)
      + acl                         = (known after apply)
      + arn                         = (known after apply)
      + bucket                      = "latam-sustainability-datalake"
      + bucket_domain_name          = (known after apply)
      + bucket_namespace            = (known after apply)
      + bucket_prefix               = (known after apply)
      + bucket_region               = (known after apply)
      + bucket_regional_domain_name = (known after apply)
      + force_destroy               = false
      + hosted_zone_id              = (known after apply)
      + id                          = (known after apply)
      + object_lock_enabled         = (known after apply)
      + policy                      = (known after apply)
      + region                      = "us-east-1"
      + request_payer               = (known after apply)
      + tags_all                    = (known after apply)
      + website_domain              = (known after apply)
      + website_endpoint            = (known after apply)

      + cors_rule (known after apply)

      + grant (known after apply)

      + lifecycle_rule (known after apply)

      + logging (known after apply)

      + object_lock_configuration (known after apply)

      + replication_configuration (known after apply)

      + server_side_encryption_configuration (known after apply)

      + versioning (known after apply)

      + website (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

──────────────────────────────────────────────────────────────────────────────
──

Note: You didn't use the -out option to save this plan, so Terraform can't
```

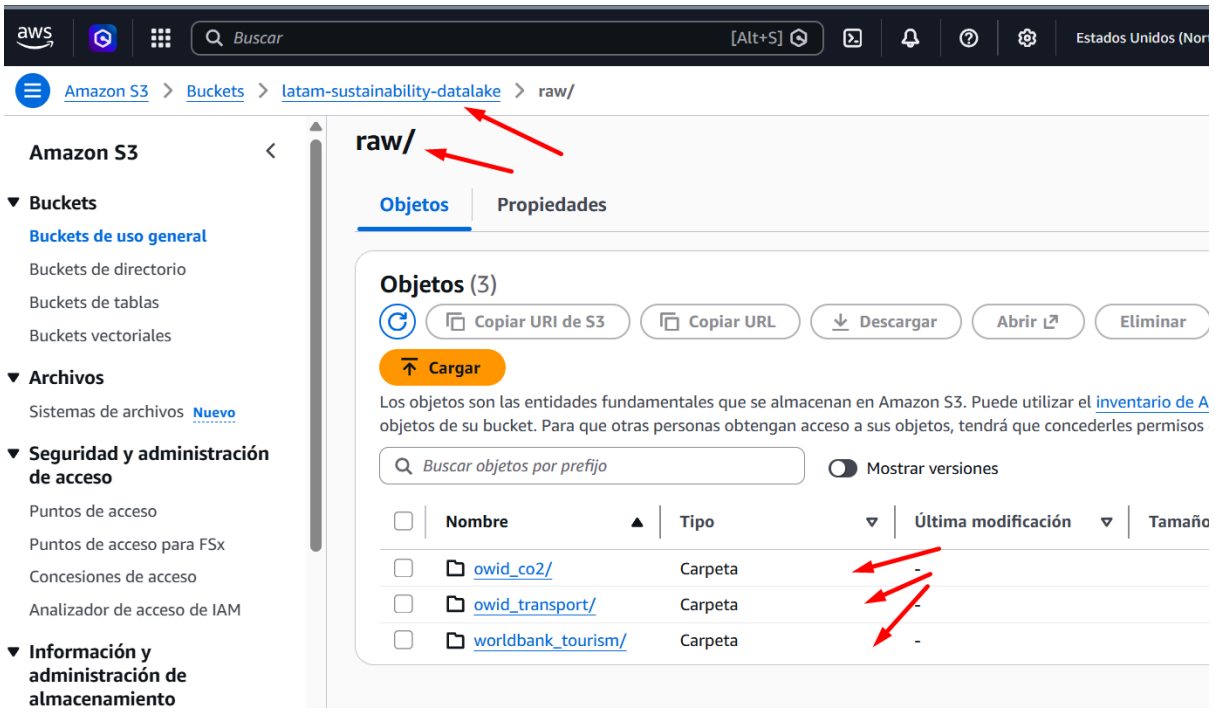
```
guarantee to take exactly these actions if you run "terraform apply" now.
```



Una vez creadas todas las carpetas, al correr terraform apply

```
$ terraform apply
module.s3.aws_s3_bucket.datalake: Refreshing state... [id=latam-sustainability-d
atalake]
aws_budgets_budget.zero_budget: Refreshing state... [id=132425676374:zero-budget
]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # module.s3.aws_s3_bucket_versioning.datalake will be created
  + resource "aws_s3_bucket_versioning" "datalake" {
      + bucket = "latam-sustainability-datalake"
      + id     = (known after apply)
      + region = "us-east-1"

      + versioning_configuration {
          + mfa_delete = (known after apply)
          + status     = "Enabled"
        }
    }

  # module.s3.aws_s3_object.bronze_co2_emissions_prefix will be created
  + resource "aws_s3_object" "bronze_co2_emissions_prefix" {
      + acl                    = (known after apply)
      + arn                    = (known after apply)
      + bucket                 = "latam-sustainability-datalake"
      + bucket_key_enabled     = (known after apply)
      + checksum_crc32         = (known after apply)
      + checksum_crc32c        = (known after apply)
      + checksum_crc64nvme     = (known after apply)
      + checksum_sha1          = (known after apply)
      + checksum_sha256        = (known after apply)
      + content_type           = (known after apply)
      + etag                   = (known after apply)
      + force_destroy          = false
      + id                     = (known after apply)
      + key                    = "bronze/co2_emissions/"
      + kms_key_id             = (known after apply)
      + region                 = "us-east-1"
      + server_side_encryption = (known after apply)
      + storage_class          = (known after apply)
      + tags_all               = (known after apply)
      + version_id             = (known after apply)
        # (1 unchanged attribute hidden)
```

Se visualizan en S3:



IAM

A continuación se crean los grupos y usuarios. Se genera el terraform plan

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

budget_name = "zero-budget"
data_engineers_group_name = "data-engineers"
datalake_bucket_name = "latam-sustainability-datalake"
project_managers_group_name = "project-manager"

Mariana@LAPTOP-KUTGC5I3 MINGW64 ~/Dropbox/Cursos/SoyHenry/DataEngineering/Modulo06-ProyectoFinal/
to-Latam-Turismo-Sustentable/infra (main)
$
```

data-engineers

data-engineers | April 04, 2026, 00:07 (

Personas (4) | Permisos | Access Advisor

Personas de este grupo (4)

Un usuario de IAM es una entidad que se crea en AWS para representar a la pers

Buscar

Nombre de usuario
adrian.sosa
luis.buruato
mariana.gil
martin.tedesco

data-engineers

data-engineers | April 04, 2026, 00:07 (UTC-03:00)

Personas (4) | Permisos | Access Advisor

Políticas de permisos (4) Información

Puede asociar hasta 10 políticas administradas.

Filtrar por Tipo

Buscar | Todos los tipos

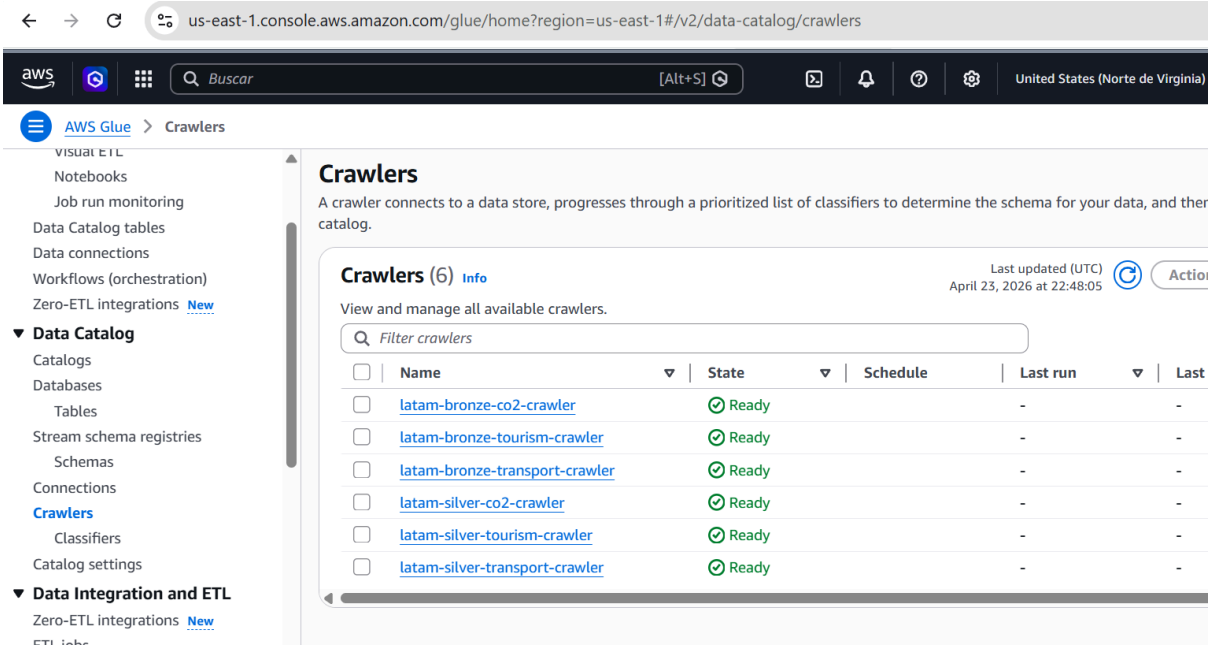
Nombre de la política	Tipo
AmazonAthenaFullAccess	Administrada por AWS
AmazonEC2FullAccess	Administrada por AWS
AmazonS3FullAccess	Administrada por AWS
AWSGlueConsoleFullAccess	Administrada por AWS

GLUE

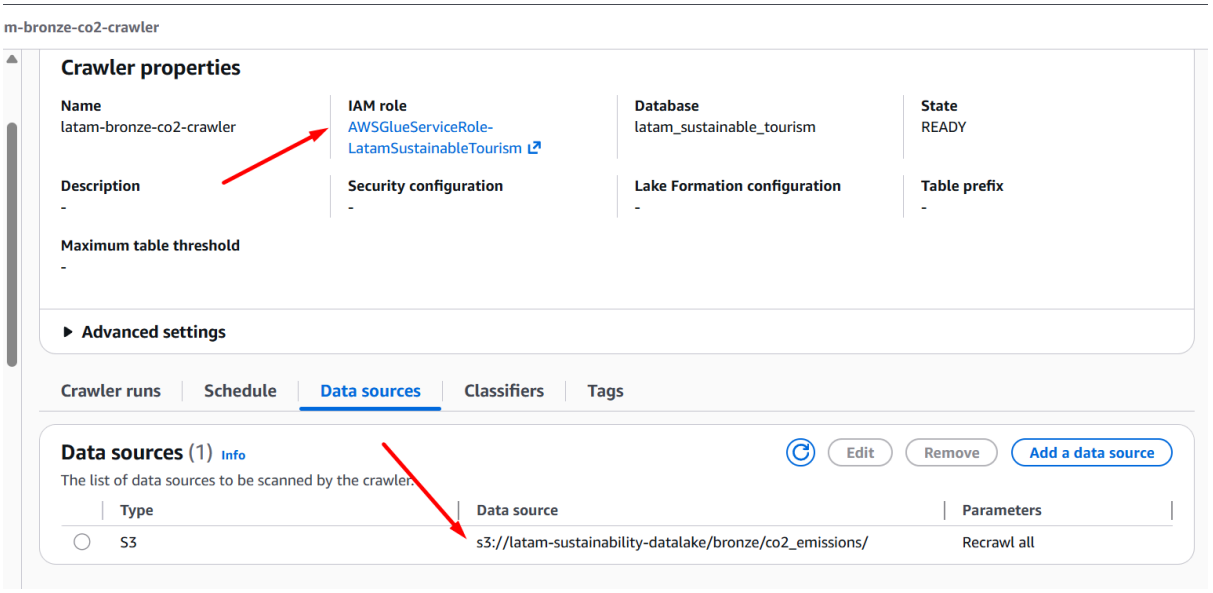
Se crea el módulo en Terraform con los roles, con la decisión de separar en bronze y silver y en co2, tourism y transport

```
Apply complete! Resources: 6 added, 0 changed, 1 destroyed.

Outputs:

bronze_co2_crawler_name = "latam-bronze-co2-crawler"
bronze_tourism_crawler_name = "latam-bronze-tourism-crawler"
bronze_transport_crawler_name = "latam-bronze-transport-crawler"
budget_name = "zero-budget"
data_engineers_group_name = "data-engineers"
datalake_bucket_name = "latam-sustainability-datalake"
glue_database_name = "latam_sustainable_tourism"
glue_service_role_arn = "arn:aws:iam::132425676374:role/AWSGlueServiceRole-Latam
SustainableTourism"
glue_service_role_name = "AWSGlueServiceRole-LatamSustainableTourism"
project_managers_group_name = "project-manager"
silver_co2_crawler_name = "latam-silver-co2-crawler"
silver_tourism_crawler_name = "latam-silver-tourism-crawler"
silver_transport_crawler_name = "latam-silver-transport-crawler"
```

Se verifica en AWS la creación de los crawlers:



Ingreso a uno de los crawlers y verifico que el rol y el path objetivo sean correctos:



Evento automatizacion

infra/eventbridge.tf

```
resource "aws_cloudwatch_event_rule" "monthly_ingestion" {
  name                = "monthly-ingestion-trigger"
  description         = "Ejecuta la ingesta de datos mensualmente"
  schedule_expression = "cron(0 0 1 * ? *)" # 1ro de cada mes a las 00:00 UTC
}
```

Target → Lambda

```
resource "aws_cloudwatch_event_target" "lambda_target" {
  rule      = aws_cloudwatch_event_rule.monthly_ingestion.name
  target_id = "lambda-ingestion"
  arn       = aws_lambda_function.ingestion_lambda.arn
}
```

Permiso para que EventBridge invoque Lambda

```
resource "aws_lambda_permission" "allow_eventbridge" {
  statement_id  = "AllowExecutionFromEventBridge"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.ingestion_lambda.function_name
  principal     = "events.amazonaws.com"
```

```
  source_arn    = aws_cloudwatch_event_rule.monthly_ingestion.arn
}
```

Lambda (por si no lo tenés completo)

infra/modules/lambda/lambda.tf

```
resource "aws_lambda_function" "ingestion_lambda" {
  function_name = "latam-ingestion-lambda"
  role          = aws_iam_role.lambda_exec.arn
  handler       = "lambda_handler.lambda_handler"
  runtime       = "python3.11"

  filename         = "${path.module}/lambda_package.zip"
  source_code_hash = filebase64sha256("${path.module}/lambda_package.zip")

  timeout = 900  # 15 minutos
  memory_size = 1024

  environment {
    variables = {
      ENV = "prod"
    }
  }
}
```

5. IAM Role para Lambda

infra/modules/lambda/iam.tf

```
resource "aws_iam_role" "lambda_exec" {
  name = "lambda-exec-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "lambda.amazonaws.com"
        }
      }
    ]
  })
}
```

Policy básica (logs + S3)

```
resource "aws_iam_role_policy" "lambda_policy" {
  name = "lambda-policy"
  role = aws_iam_role.lambda_exec.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "logs:CreateLogGroup",
          "logs:CreateLogStream",
          "logs:PutLogEvents"
        ]
        Resource = "*"
      },
      {
        Effect = "Allow"
        Action = [
          "s3:PutObject",
          "s3:GetObject",
          "s3:ListBucket"
        ]
        Resource = "*"
      }
    ]
  })
}
```

Test manual (bash)

```
aws lambda invoke \
  --function-name latam-ingestion-lambda \
  response.json
```

terraform — qué agregar ahora para Silver/Gold

Lo que ya tienen cubre ingesta. Para Silver y Gold necesitan agregar a los módulos existentes:

módulo glue — ampliar los crawlers actuales:

hcl# Agregar a modules/glue/main.tf

```
resource "aws_glue_crawler" "silver_co2" {
  name          = "${var.project}-silver-co2"
  role          = var.glue_role_arn
```

```
  database_name = aws_glue_catalog_database.main.name
  s3_target { path = "s3://${var.bucket}/silver/co2_emissions/" }
  schedule      = "cron(0 7 1 * ? *)"   # día 1 de cada mes, 7 AM UTC
}
# Idem para silver_tourism, silver_transport, gold_fact, gold_dim
```

módulo `lambda` — actualizar el handler para que corra Silver y Gold: El Lambda que tienen hoy corre `run_ingestion.py`. Hay que extenderlo para que llame también `run_transformation.py` y `run_gold.py` en secuencia después de la ingesta.

EventBridge → trigger Airflow DAG
Airflow DAG:
  → ingest_co2
  → ingest_transport
  → ingest_worldbank
  → bronze_to_silver